Lab 5: Event-Driven Data Processing with ECS + Step Functions

Lab Name

Event-Driven Data Processing Pipeline using ECS, Step Functions, DynamoDB and EventBridge

Lab Summary

This lab demonstrates how to build an event-driven data pipeline on AWS using containerized applications. You deploy two Python applications as ECS tasks — one for data validation and another for ETL processing — orchestrated using Step Functions. EventBridge triggers the workflow when new files arrive in S3, and processed results are stored in DynamoDB with files archived in S3.

Prerequisites

- AWS account with admin permissions
- AWS CLI configured locally
- Docker installed and running
- Source code for both applications
- Input datasets (orders.csv, order_items.csv, products.csv)
- Existing S3 bucket
- Default VPC available
- Basic understanding of ECS, IAM, and S3

Steps to Complete Lab

Task 1 — Prepare Input Data in S3

Create Folder Structure

1. Open AWS S3 console
2. Open your lab bucket
3. Create folder

   e-commerce-data/new

Upload Input Files

1. Upload the following files into the new folder

- orders.csv
- order_items.csv
- products.csv

Task 2 — Run Data Validation Application Locally

Configure AWS CLI

1. Open terminal
2. Run

aws configure

1. Enter credentials and region

Build Docker Image

1. Navigate to Docker Data Validity directory
2. Run

docker build -t ecom-data-validation .

Run Container

1. Run

docker run -d -v ~/.aws:/root/.aws ecom-data-validation

Verify Execution

1. Check container logs in Docker Desktop
2. Confirm validation results show value 1

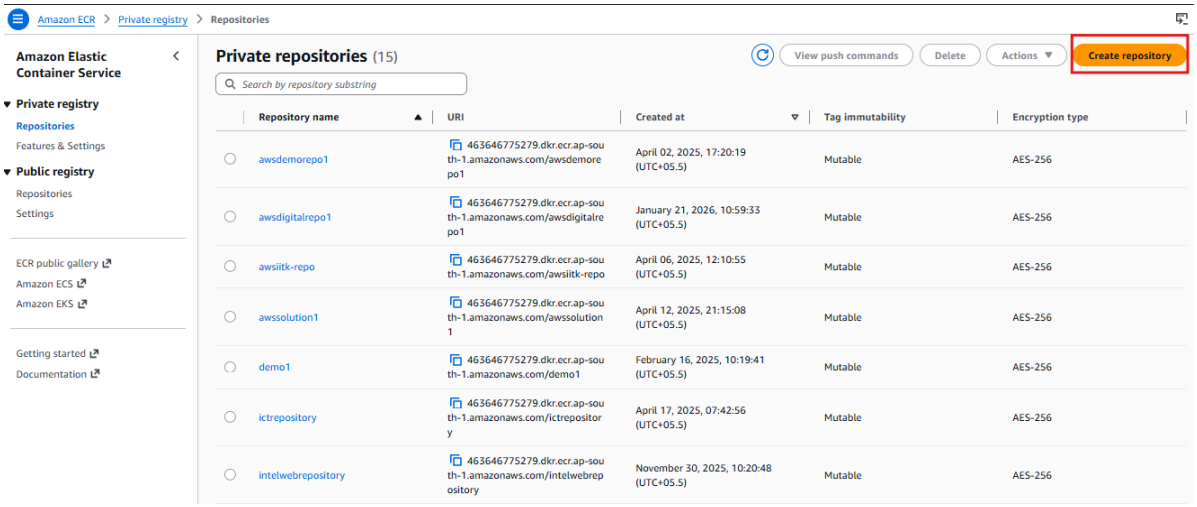
Verify File Movement

1. Check S3 bucket
2. Confirm files moved from new to respective folders

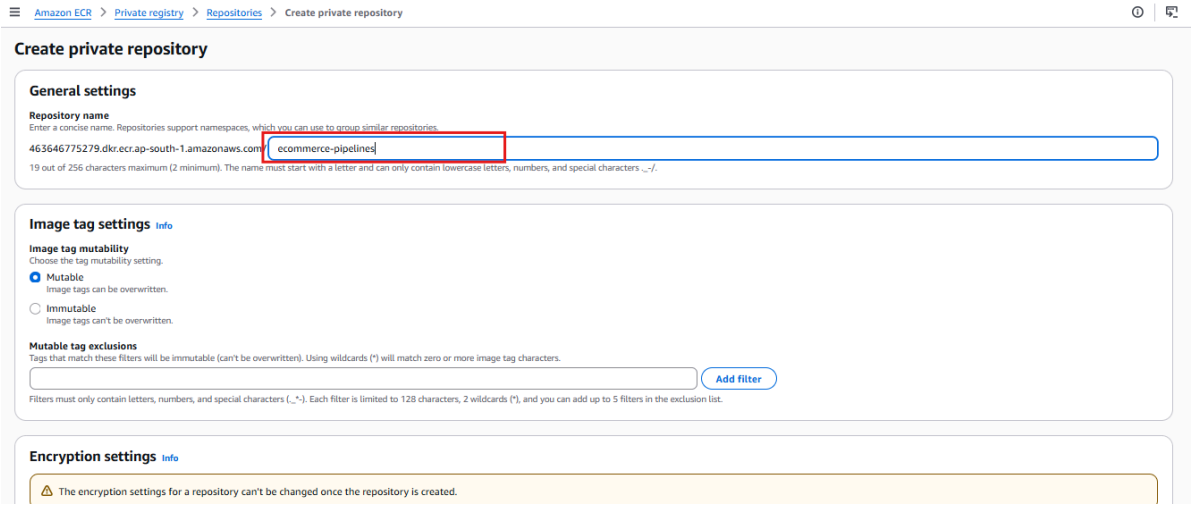
Task 3 — Push Validation Image to ECR

Create Repository

1. Open ECR console → Click on Create Repository



2. Create repository named: eg. e-commerce-pipelines



3. Click create

Authenticate Docker

1. Run login command from ECR

Tag Image

1. Run

   docker tag ecom-data-validation <ECR_URI>:latest

Push Image

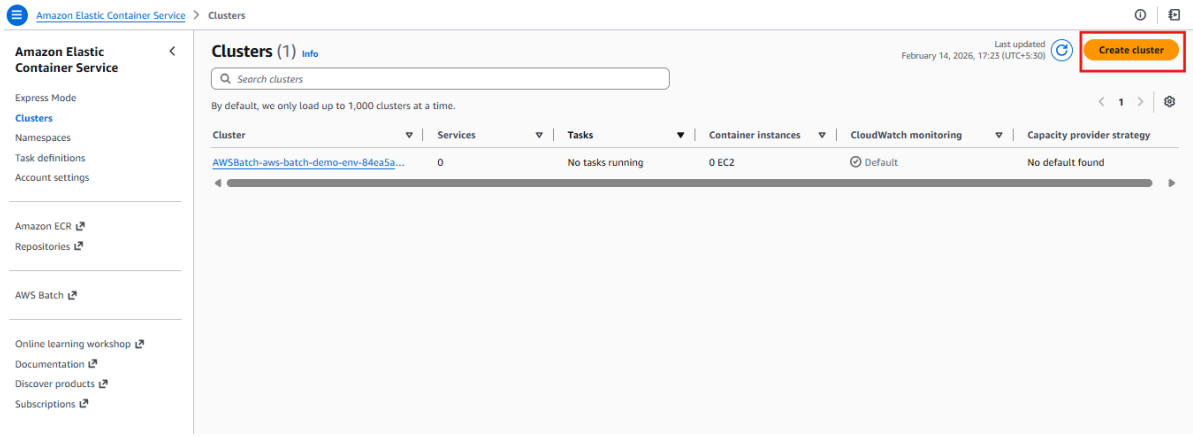
1. Run

   docker push <ECR_URI>:latest

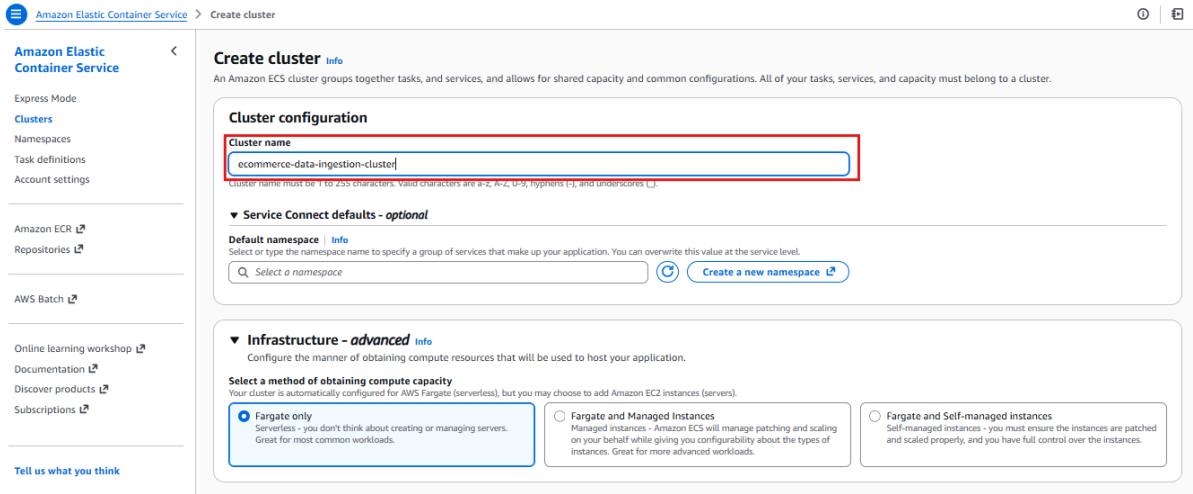
Task 4 — Deploy Validation Task on ECS

Create ECS Cluster

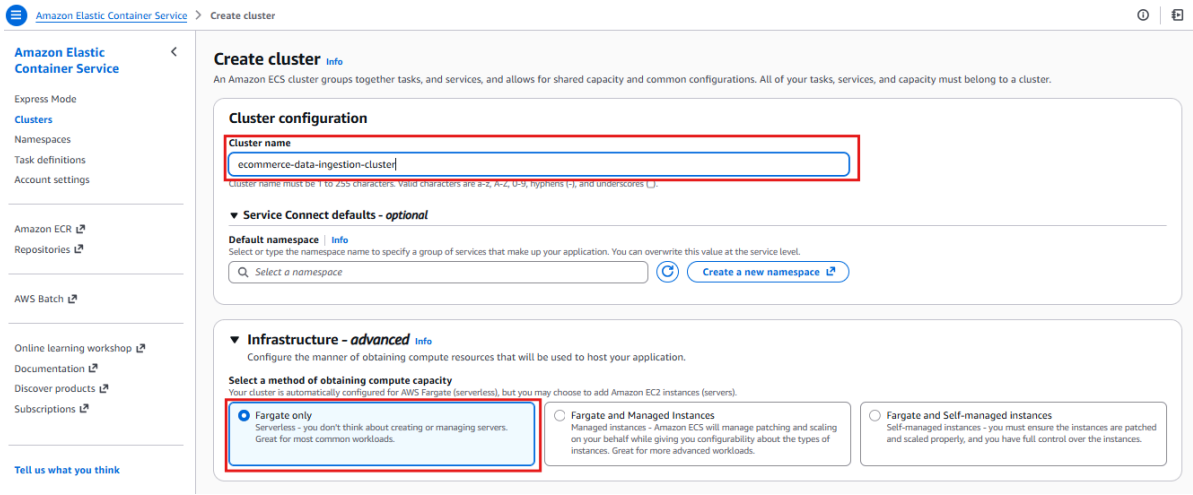
1. Open ECS console —→ Create cluster



2. Name: ecommerce-data-ingestion-cluster



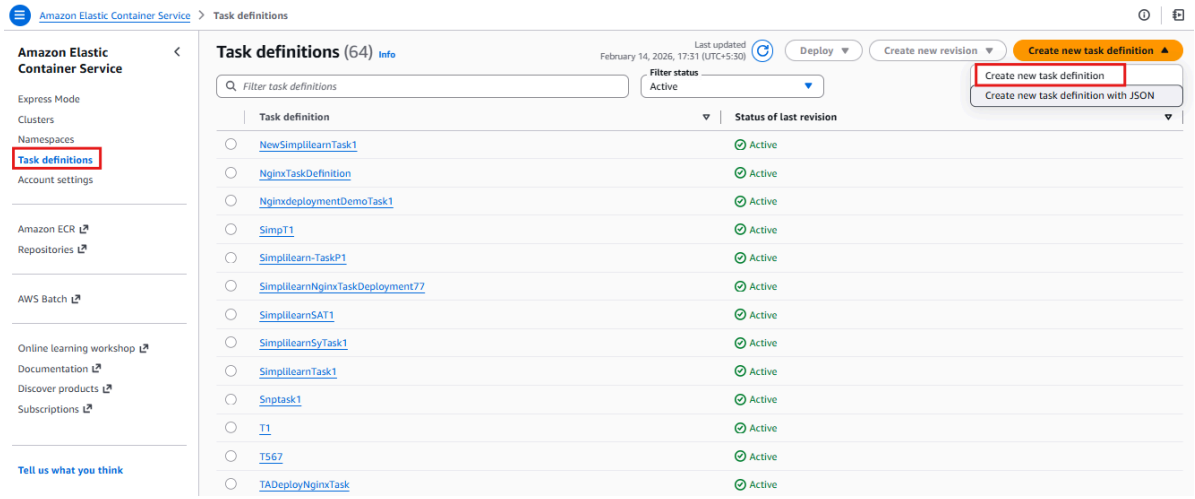
3. Infrastructure: Fargate



4. Click create

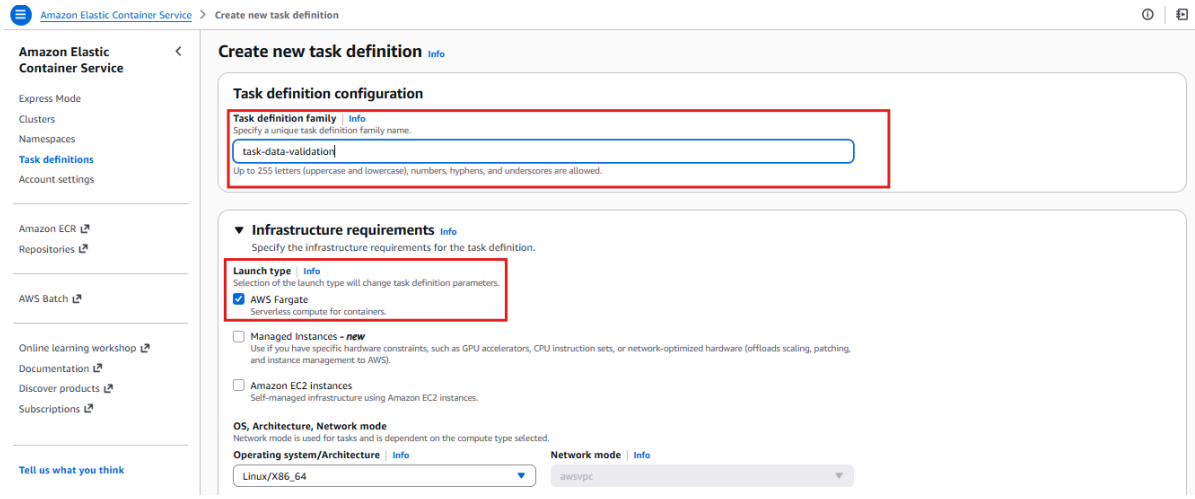
Create Task Definition

1. Create task definition



Name: task-data-validation

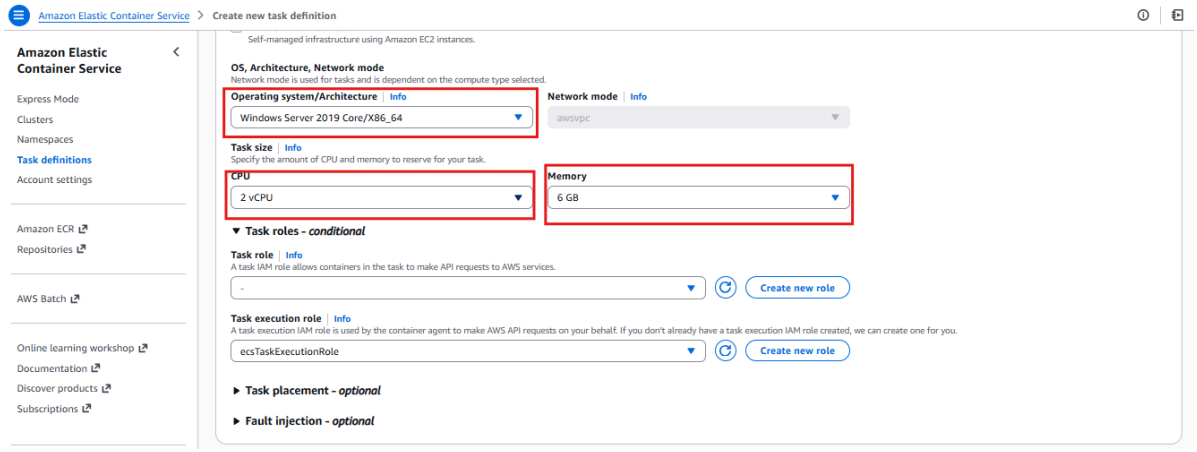
Launch type: Fargate



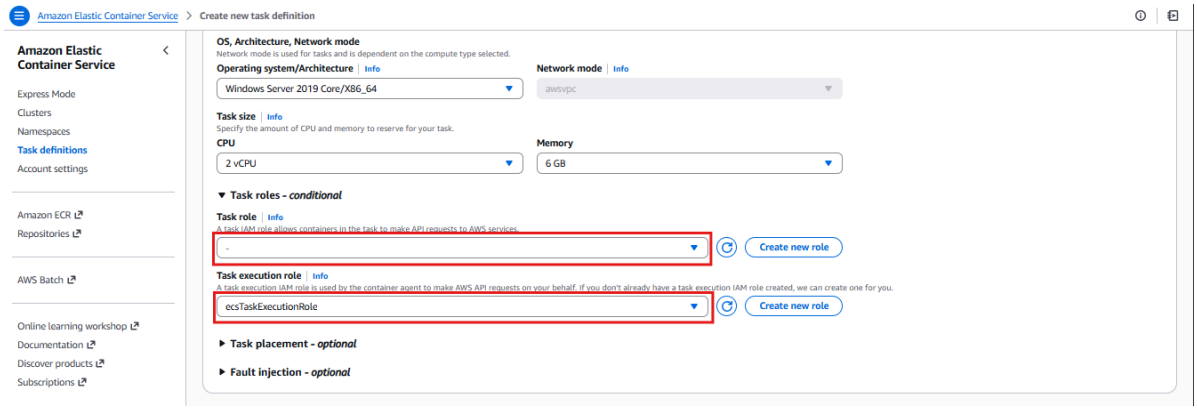
2. Configure
   Operating system/Architecture: (as per your system)

   CPU: 2 vCPU

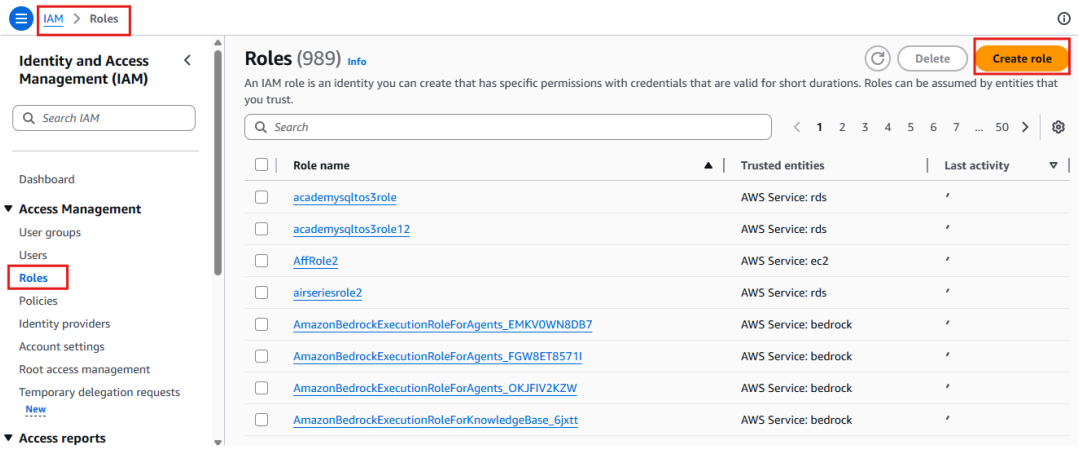
Memory: 6 GB



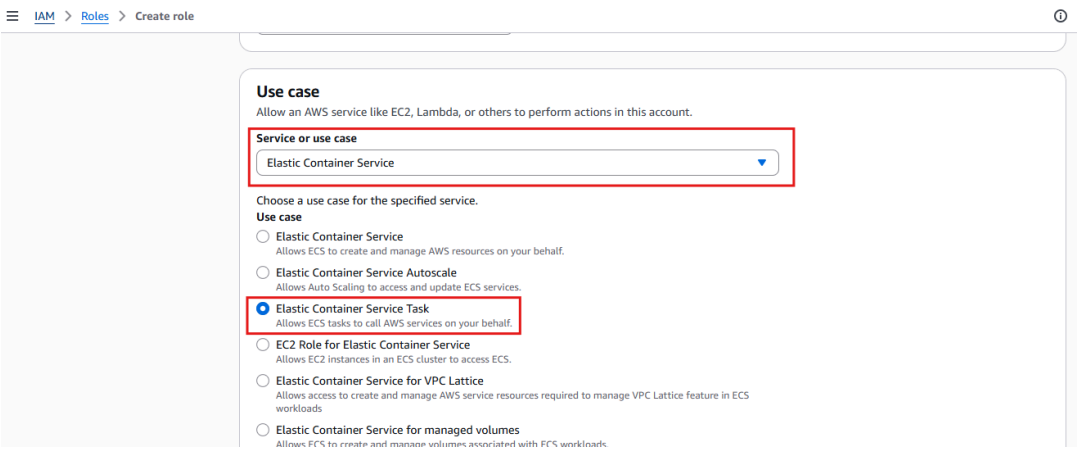
3. Select custom role for task and execution role —→ Go to IAM



1. Create IAM Role
   a. Open IAM → Create role



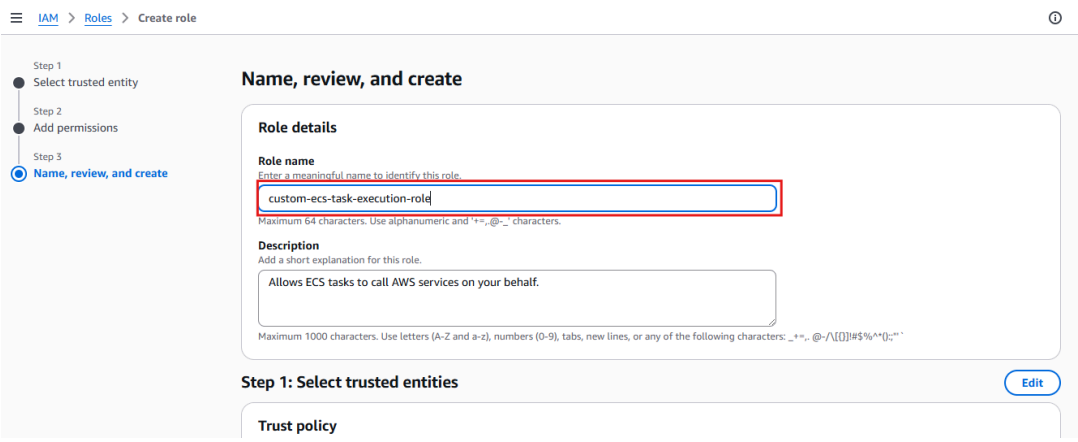
b. Use case: Elastic container service —→ Elastic container service task



c. Attach policies

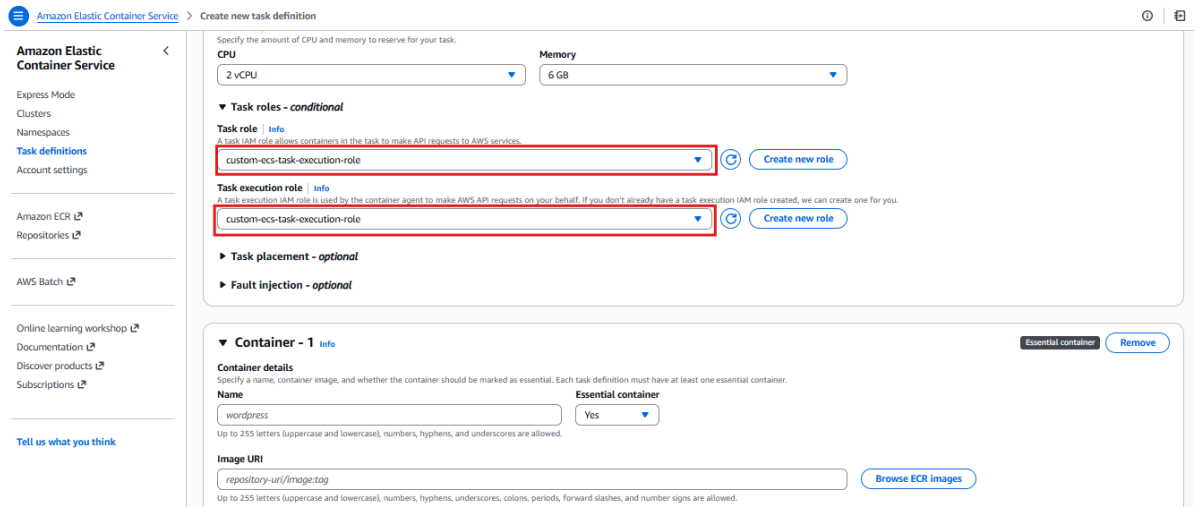
- AmazonS3FullAccess
- CloudWatchLogsFullAccess
- AmazonECSTaskExecutionRolePolicy

1. Name role: custom-ecs-task-execution-role

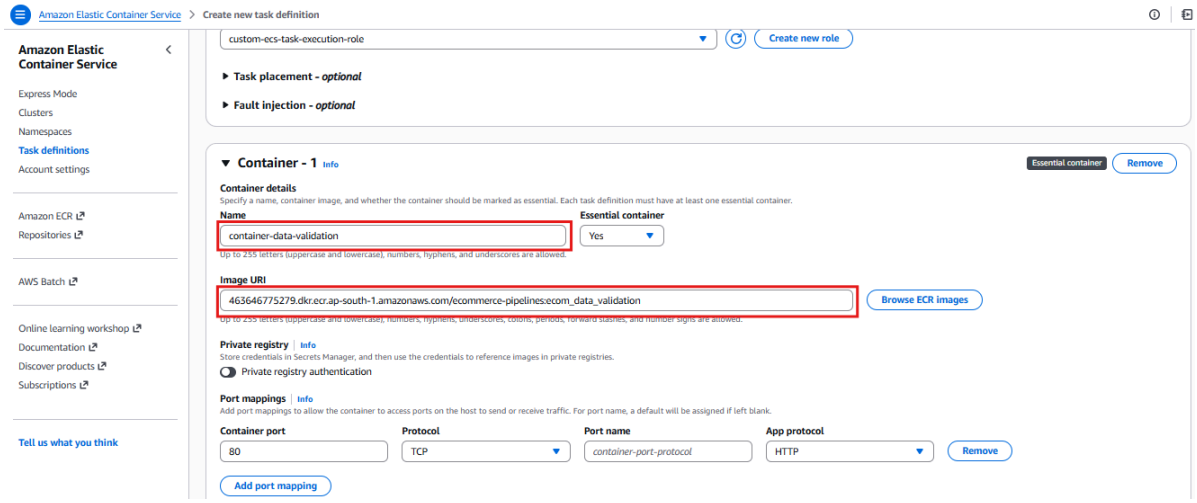


2. Click create role

4. Navigate back to ECS task creation tab —→ Select the newly created role



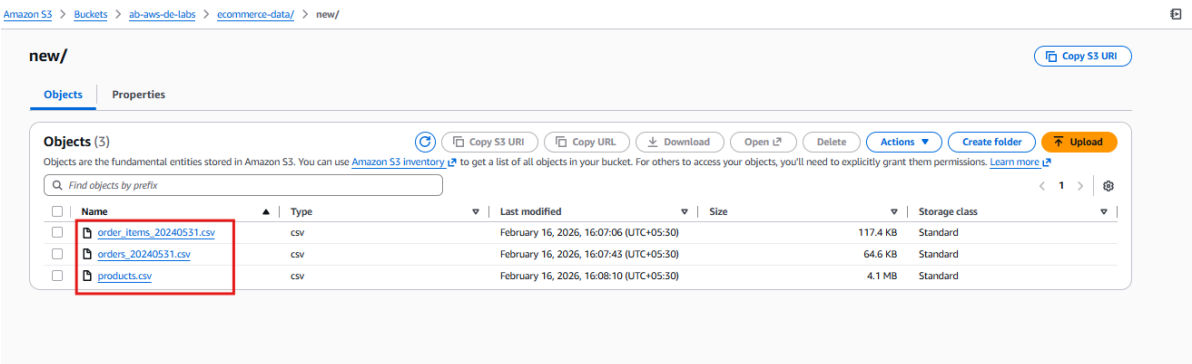
5. Under container details —→ Add container name and ECR image URI



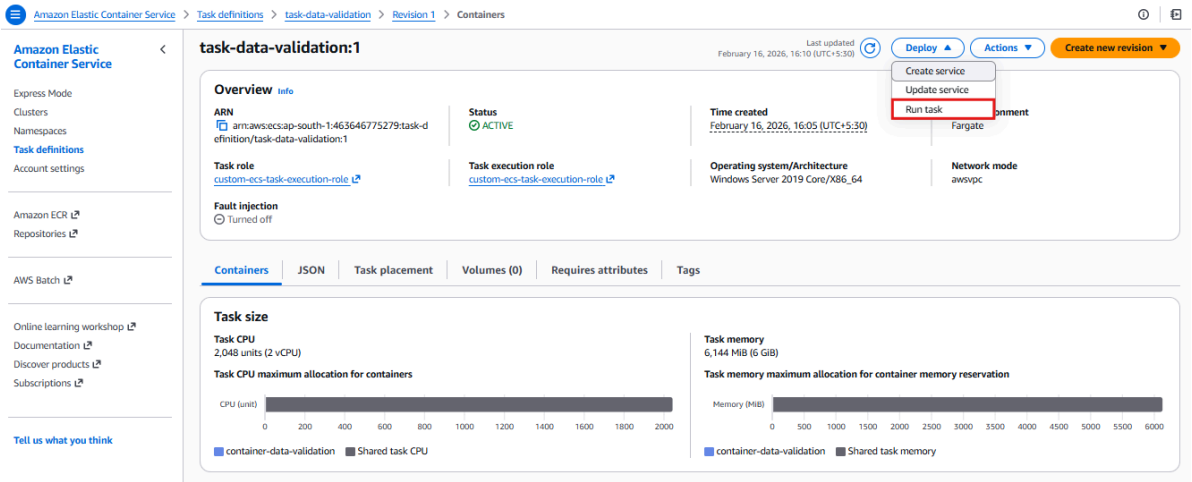
6. Click Create task

Run Task

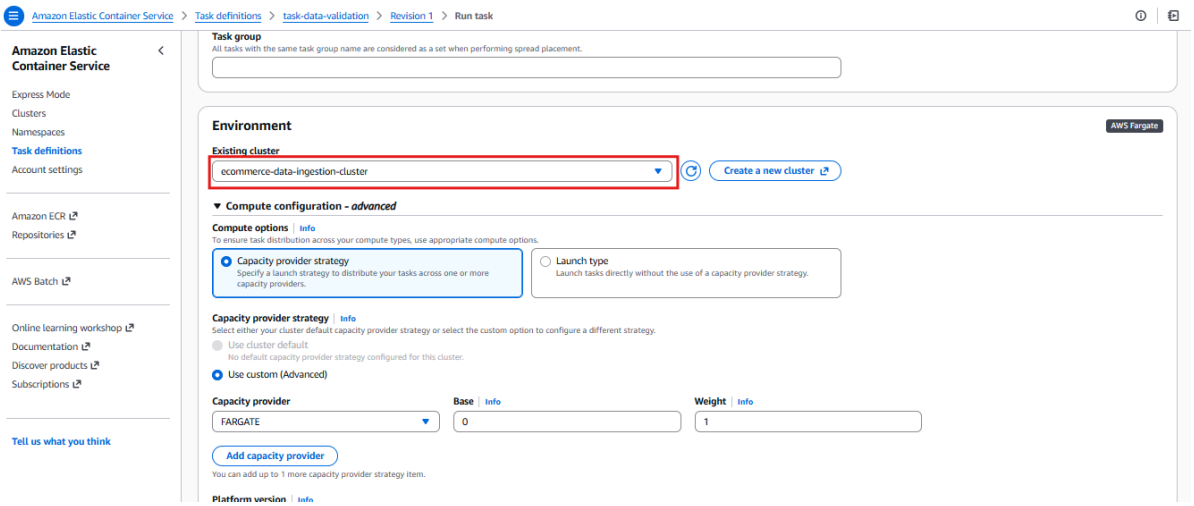
1. Upload datasets again to new folder



2. Run ECS task



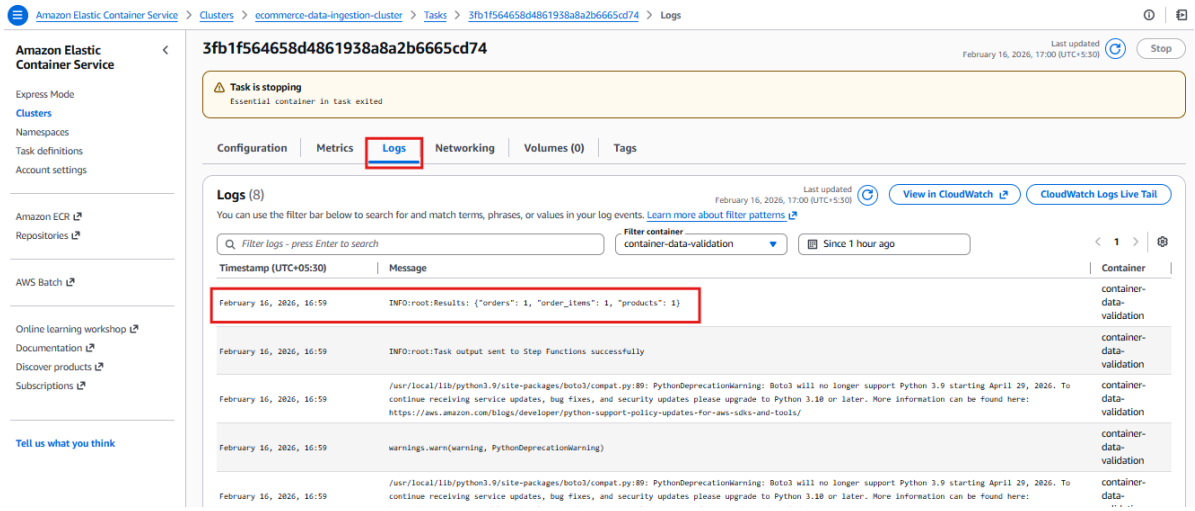
3. Select the cluster



4. Scroll down —→ Click on create

Verify Execution

1. Check task logs



2. Confirm files moved in S3

Task 5 — Run ETL Application

Build ETL Docker Image

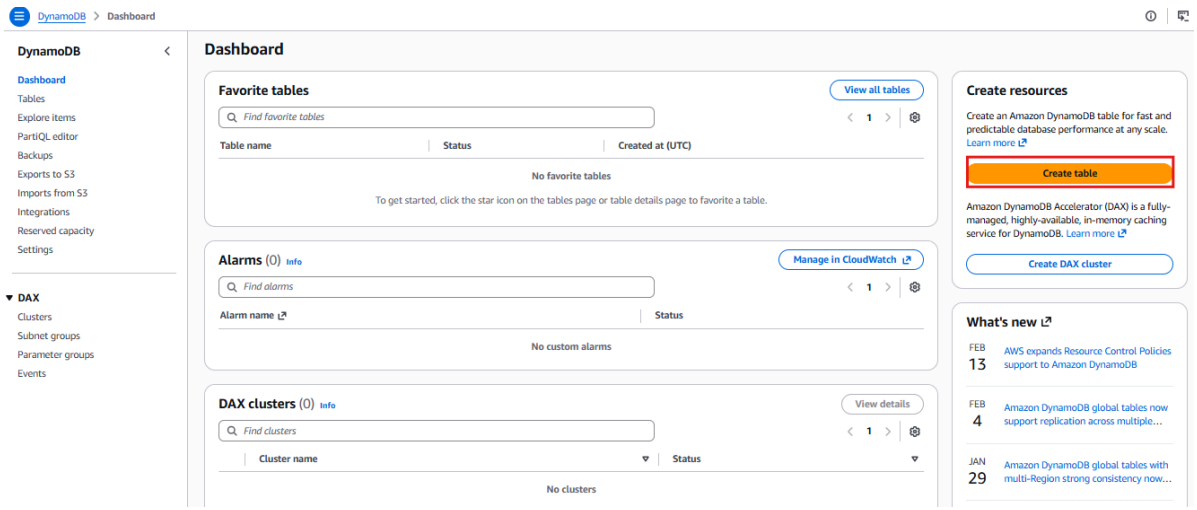
1. Navigate to Docker Dynamo ETL Wrangler directory
2. Run

   docker build -t etl-aggregations .

Create DynamoDB Tables

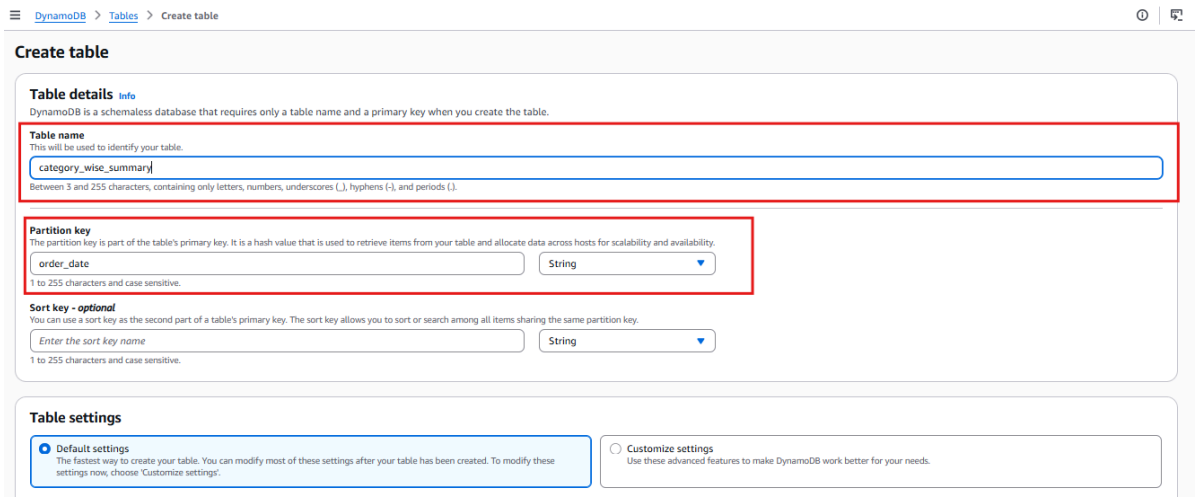
Create Category Summary Table

1. Open DynamoDB → Create table



Name: category-wise-summary

Partition key: order_date



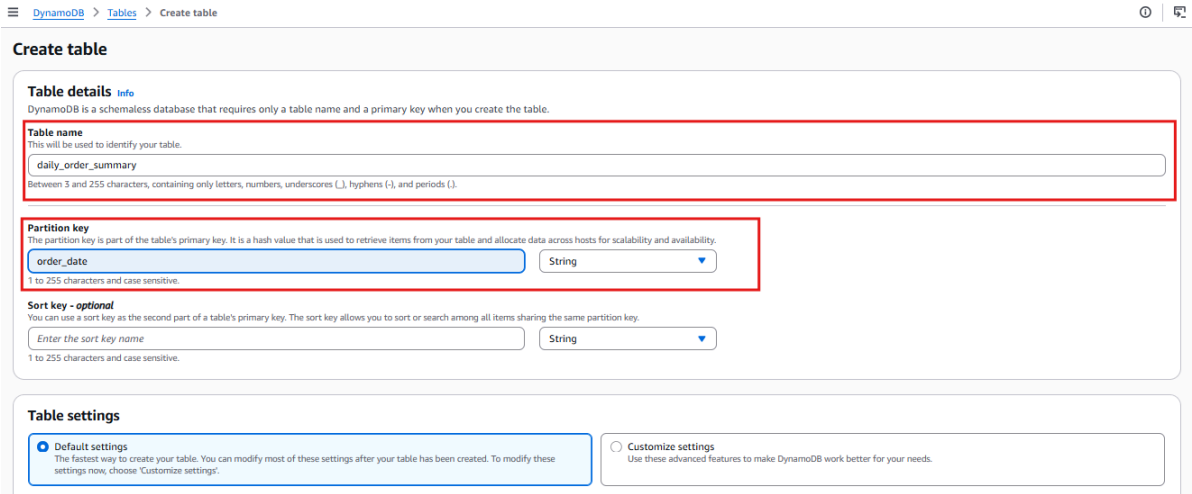
2. Scroll down —→ Click create table

Create Daily Summary Table

1. Create table

   Name: daily-order-summary

   Partition key: order_date



2. Scroll down —→ click create table

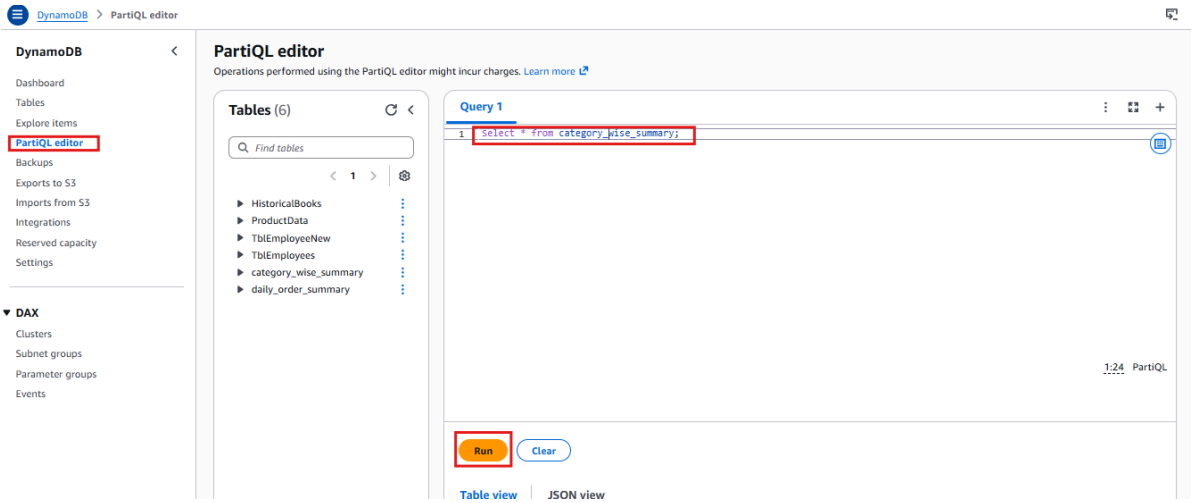
Run Container Locally

1. Run

   docker run -d -v $HOME\.aws:/root/.aws etl_aggregations

Verify Output

1. Check logs
2. Confirm records inserted into DynamoDB



Task 6 — Push ETL Image to ECR

Tag Image

1. Run

   docker tag etl-aggregations <ECR_URI>:latest

Push Image

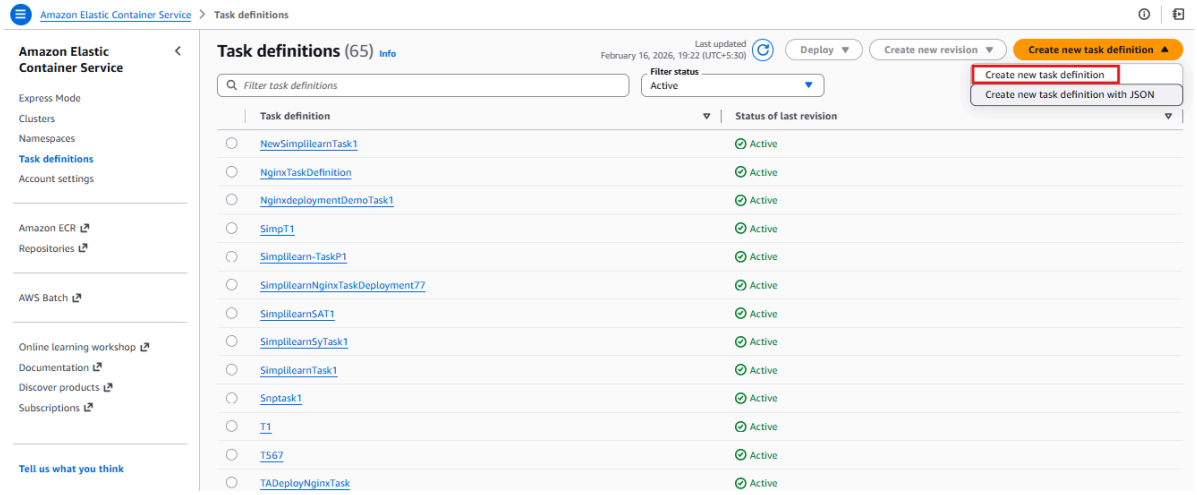
1. Run

   docker push <ECR_URI>:latest

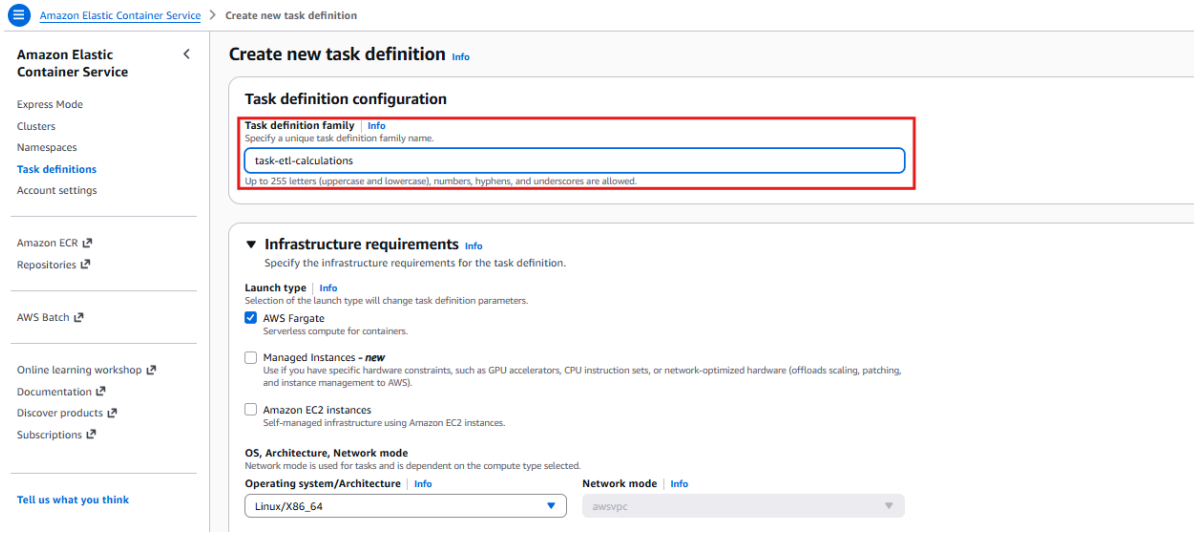
Task 7 — Deploy ETL Task on ECS

Create Task Definition

1. Create task



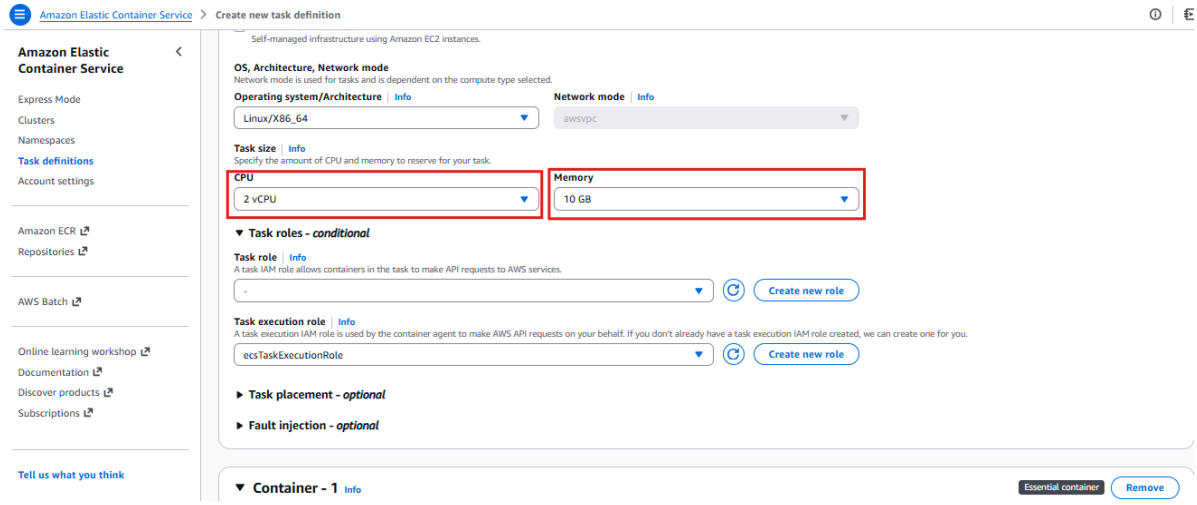
   Name: task-etl-calculations



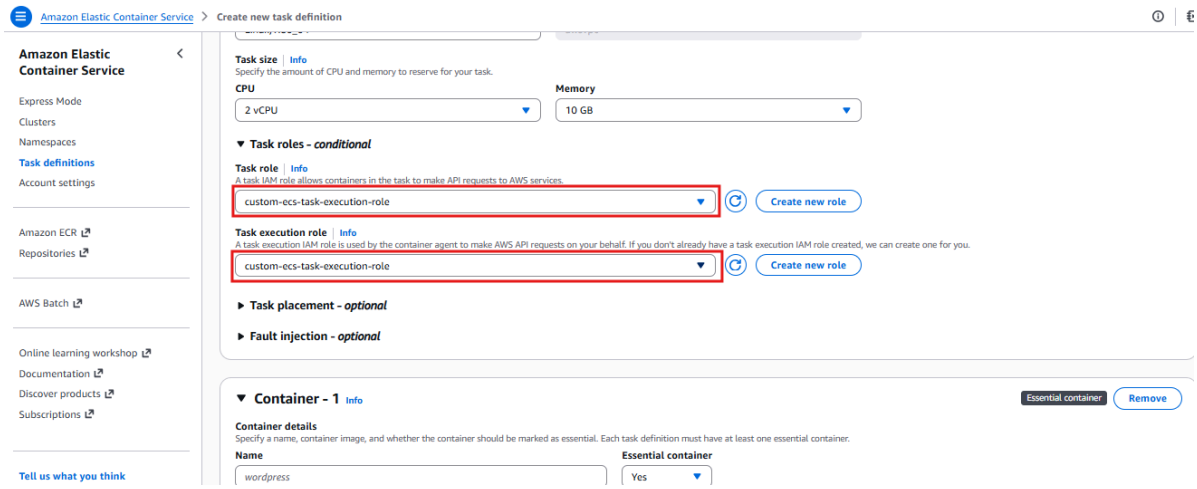
2. Configure

   CPU: 2 vCPU

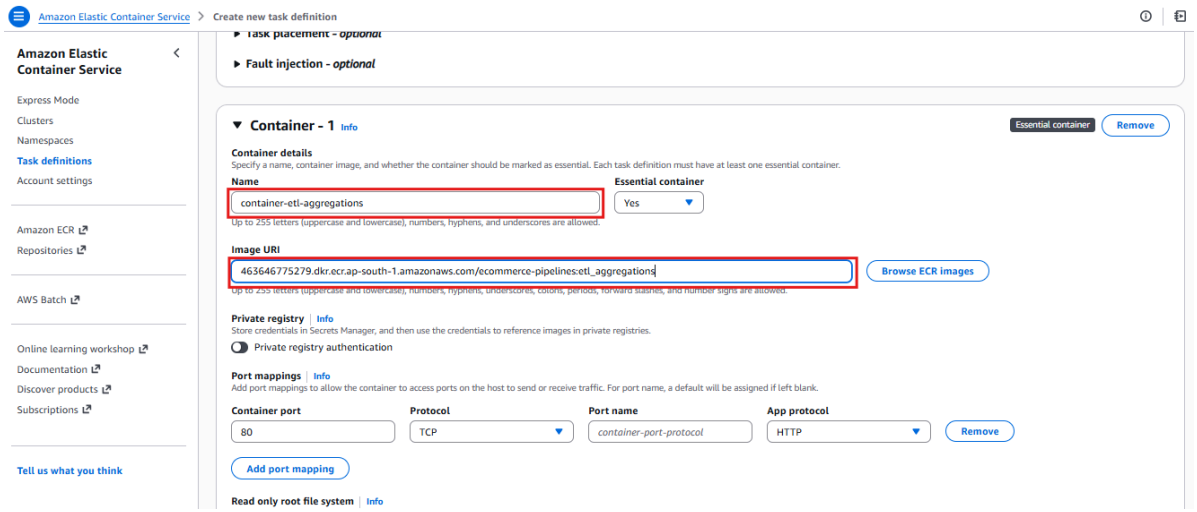
   Memory: 10 GB



3. Select custom ECS role



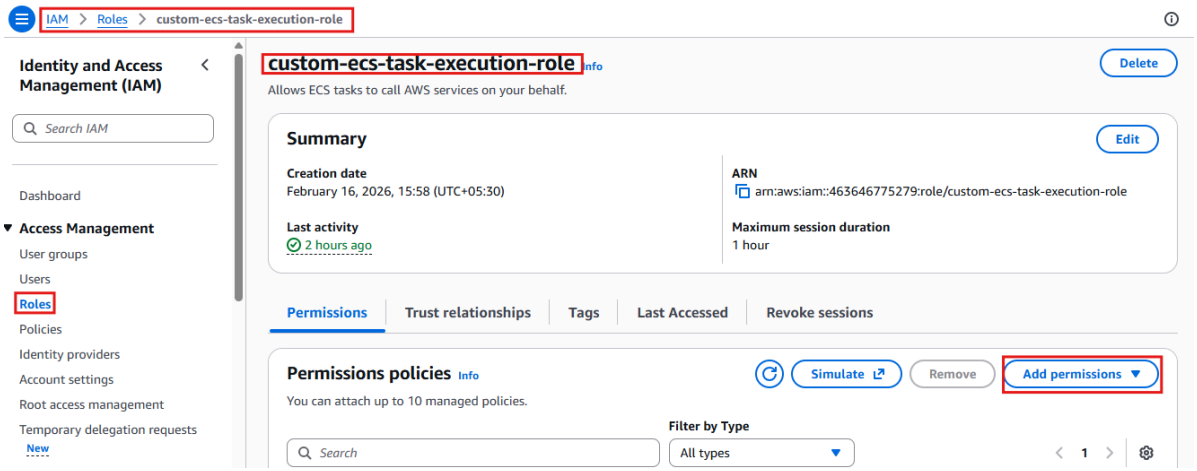
4. Add container name with ETL image URI



5. Create task

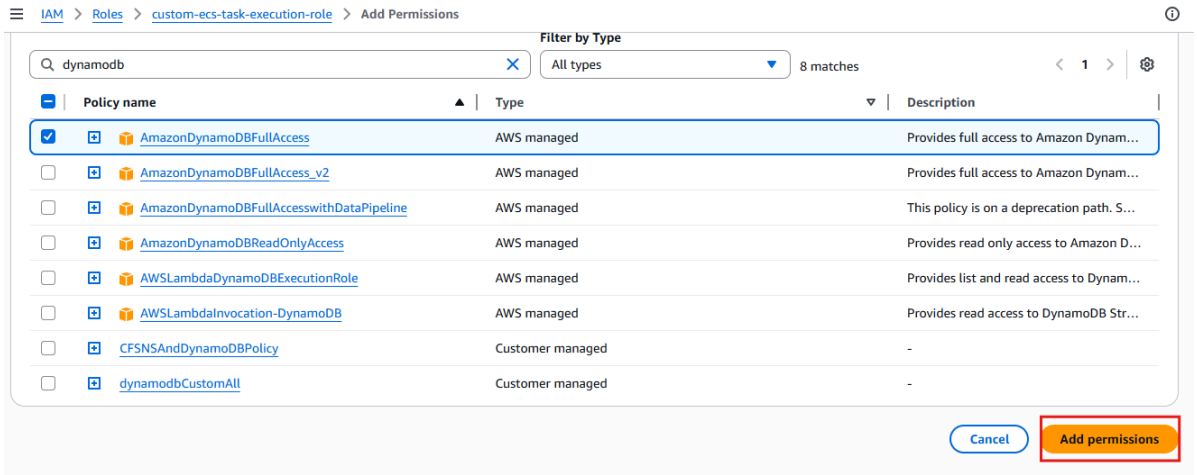
Add DynamoDB Permission

1. Open the IAM role: custom-ecs-task-execution-role —→ Add permission



2. Attach policy

   AmazonDynamoDBFullAccess

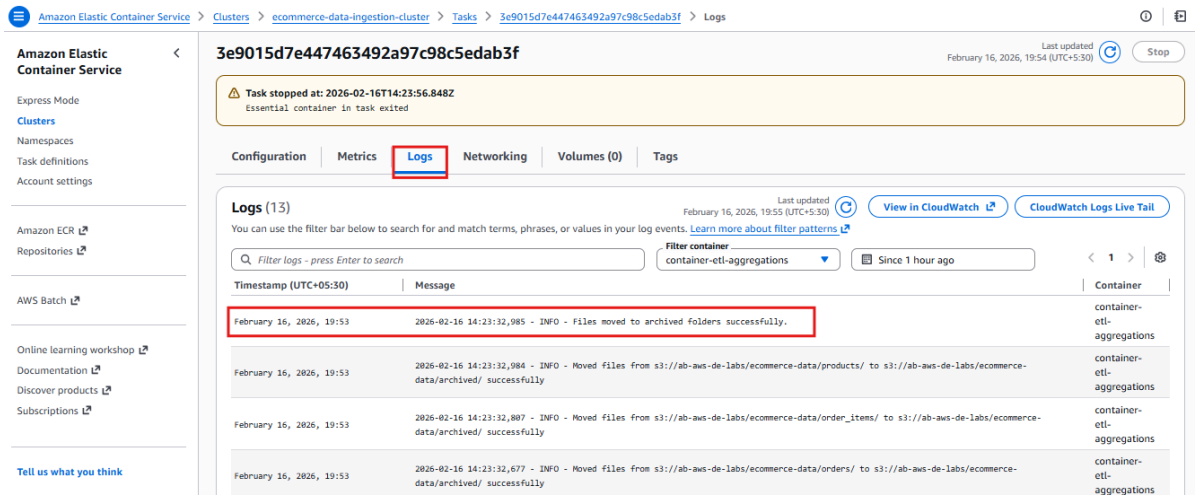


Run Task

1. Upload input files to S3
2. Run ECS task: task-data-validation
3. Run ECS task: task-etl-calculations

Verify Execution

1. Check logs
2. Confirm files moved to archive



3. Verify DynamoDB tables

Task 8 — Deploy Step Functions Workflow

Update Validation Application

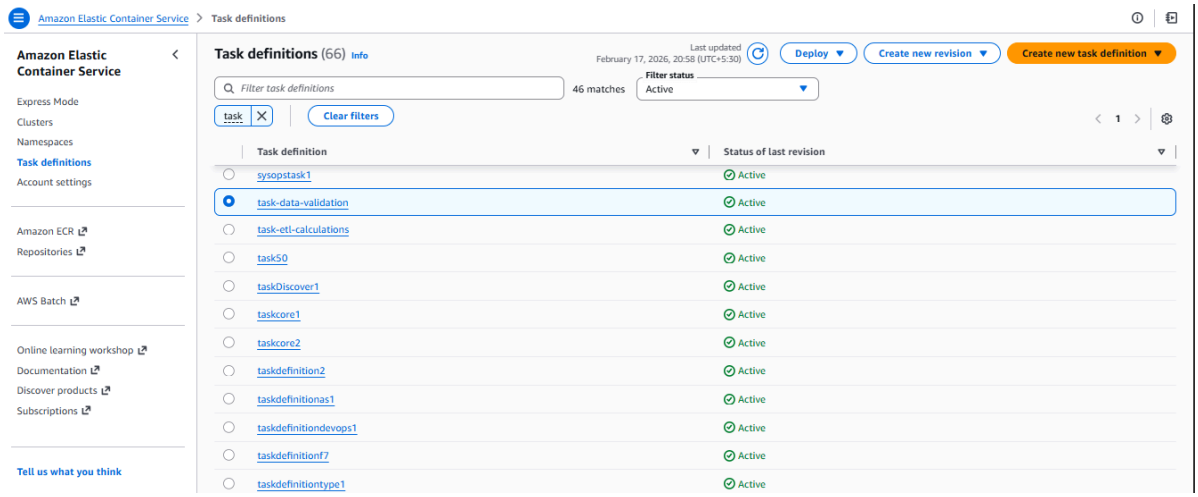
1. Uncomment Step Function token code in app.py

Rebuild and Push Image

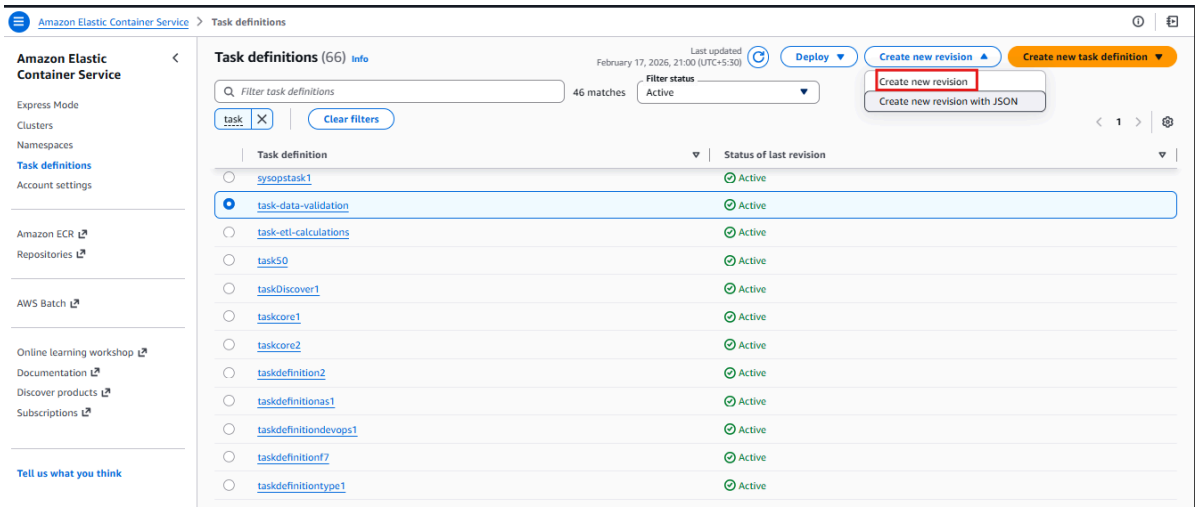
1. Rebuild Docker image
2. Tag new version
3. Push to ECR

Create New Task Revision

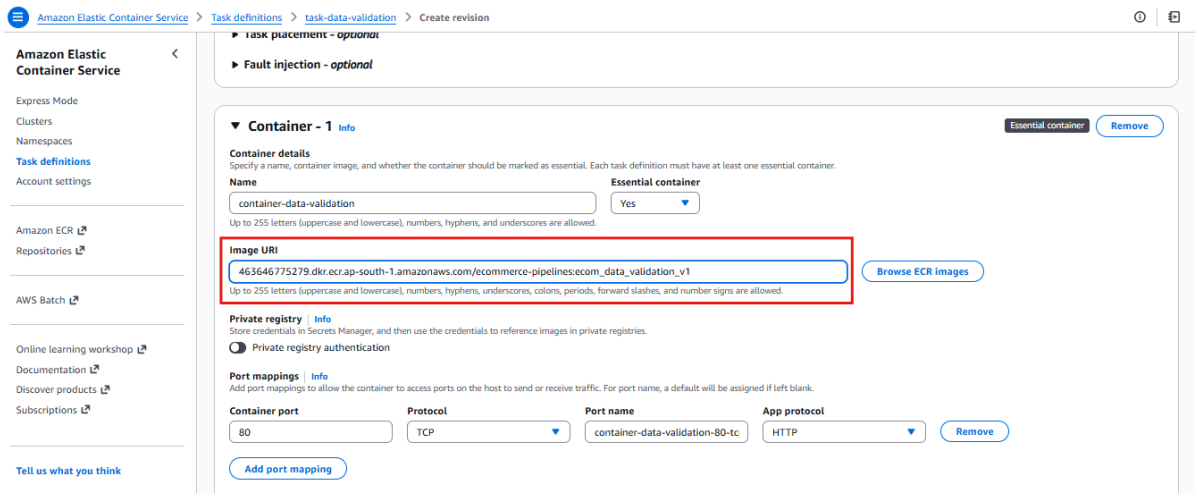
1. Open ECS task definition



2. Click on the task → Click on Create new revision

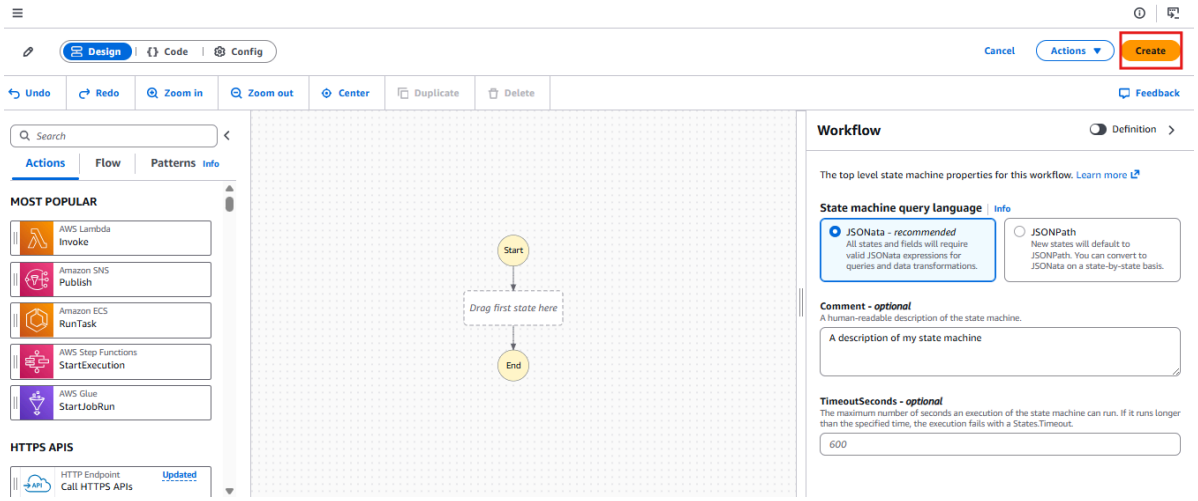


3. Update image URI —→ Click create

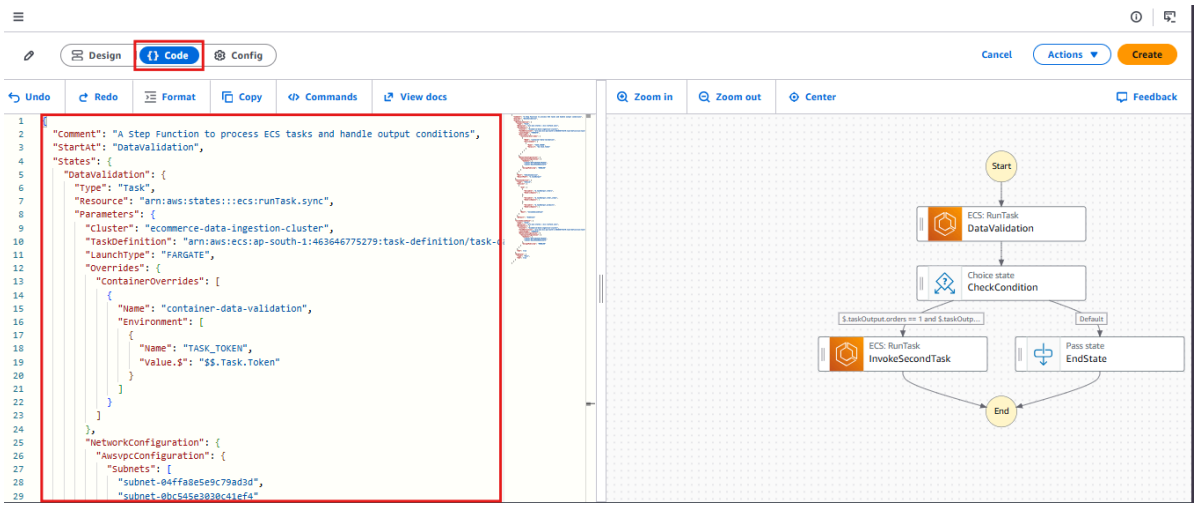


Create State Machine

1. Open Step Functions —→ Create state machine

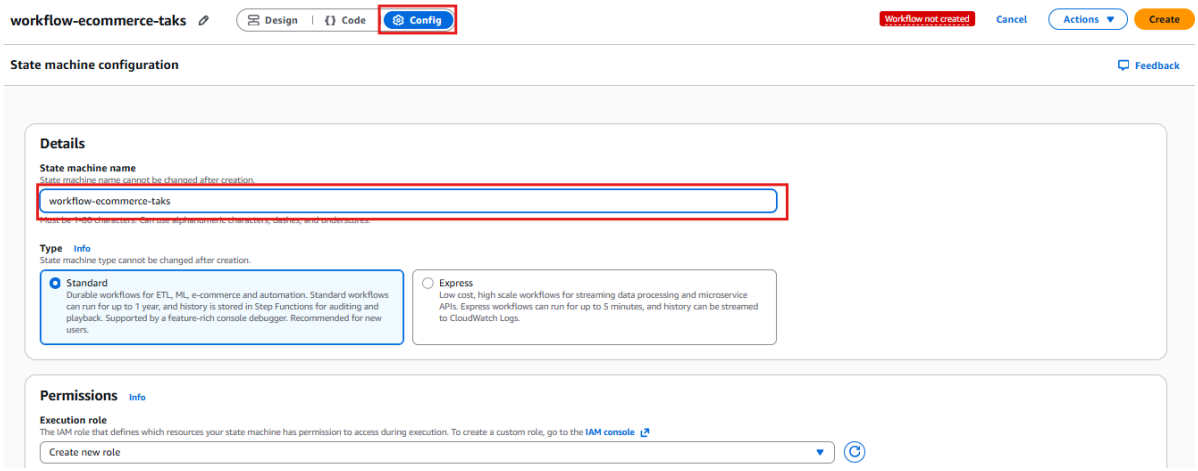


2. Click on code tab —→ Paste JSON definition code

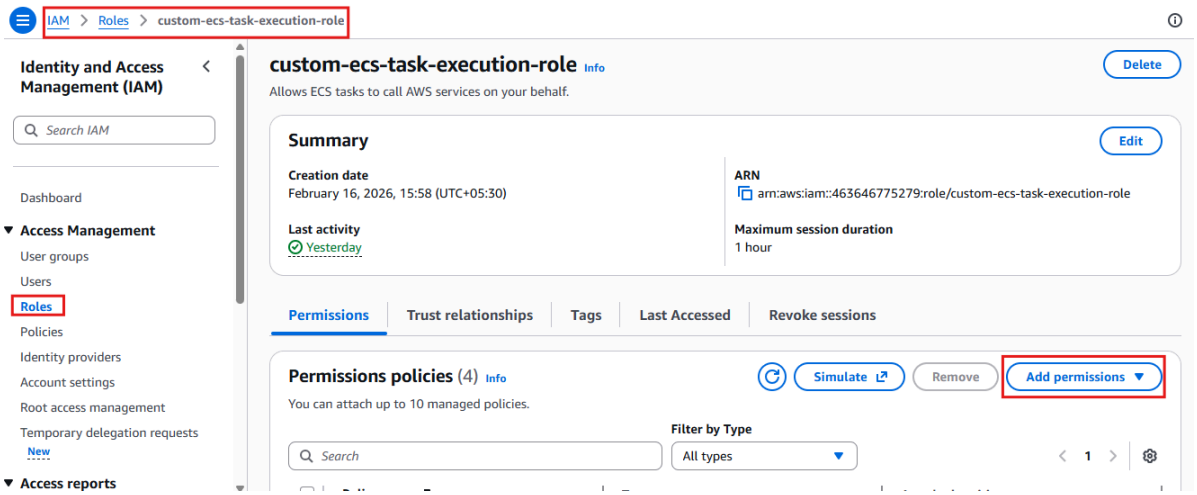


3. Update ARNs and subnet IDs

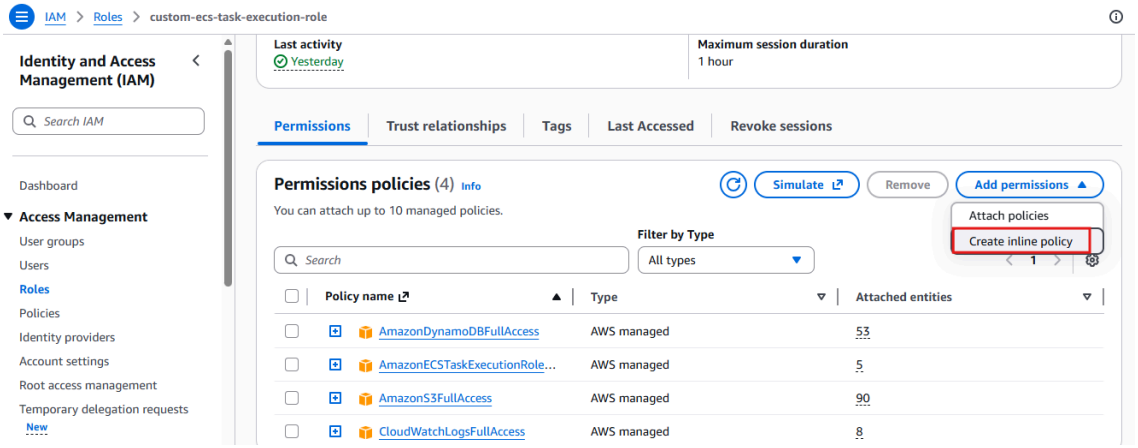
4. Click on config tab —→ give a name

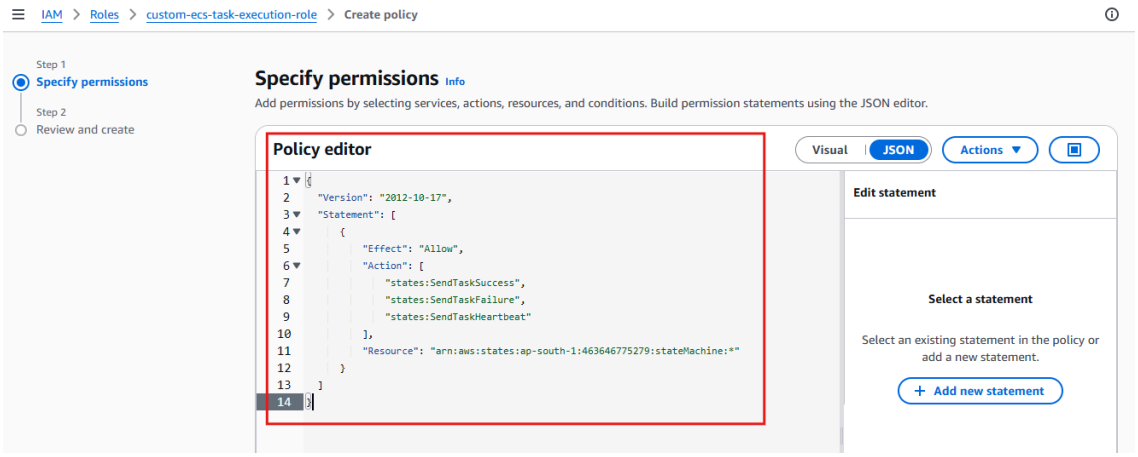


5. Go to IAM —→ Find the role created —→ Attach following policies



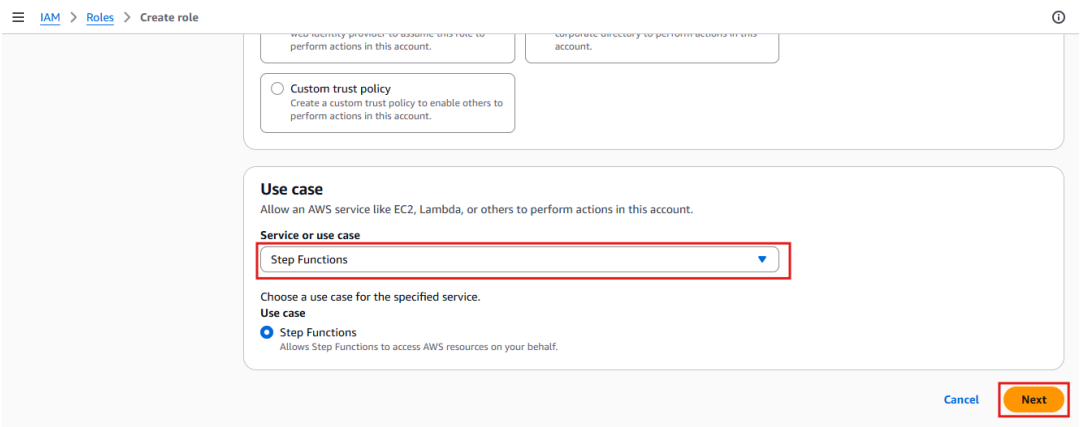
- Add inline policy from provided JSON



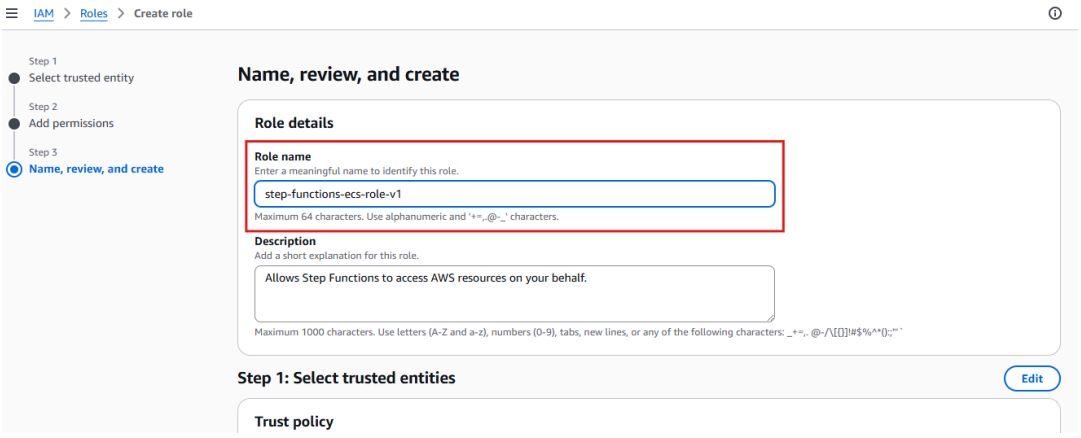


- Create new IAM role

- Select Use case: : Step functions —→ Click next

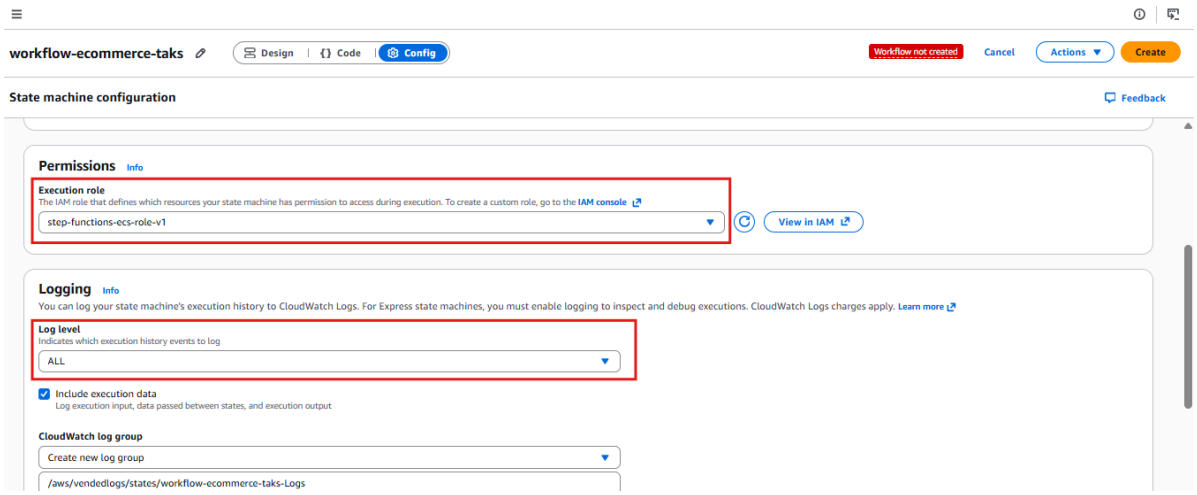


- Name the role —→ Click Create role



- Add permissions:
  - CloudWatchLogsFullAccess
  - AmazonECSFullAccess

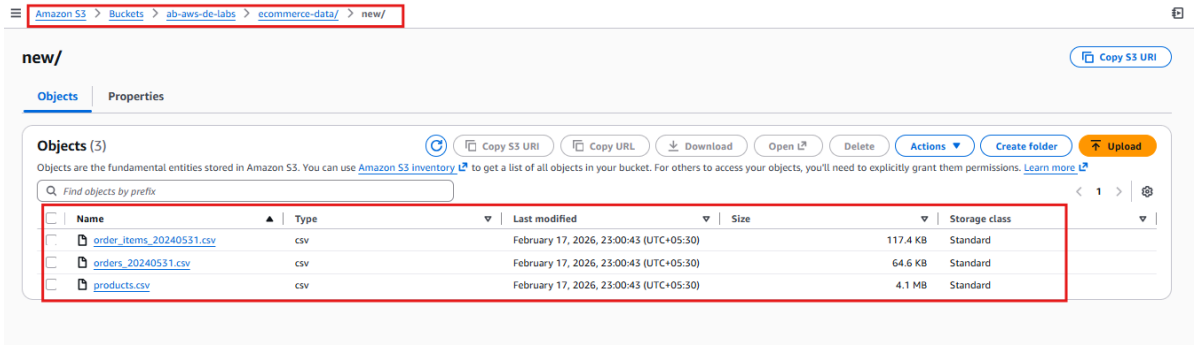
6. Get back to Steps functions creation page —→ Select the created role



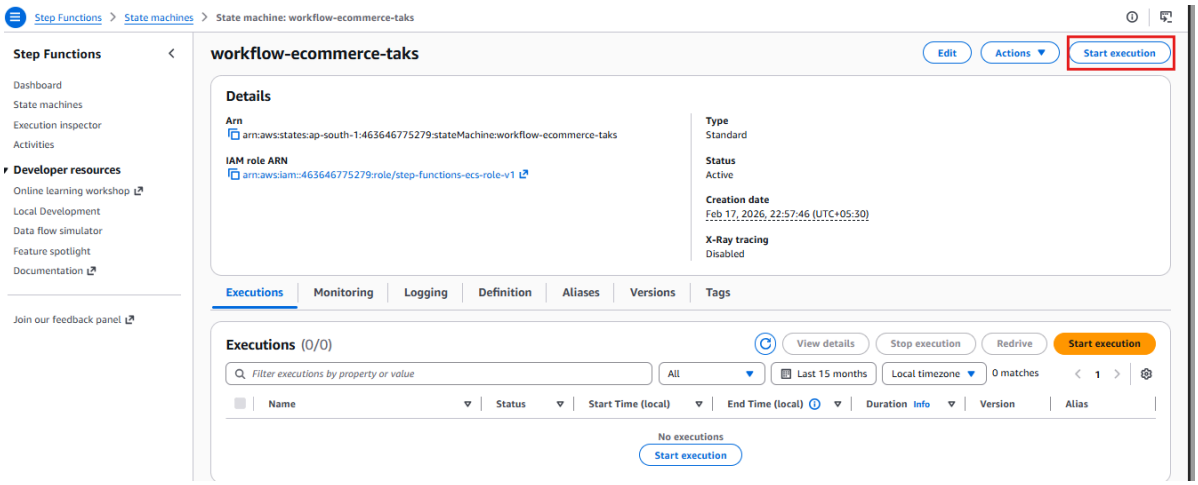
7. Click Create

Test Execution

1. Upload files to input folder: new



2. Start execution



Verify Workflow

1. Monitor execution graph
2. Confirm both ECS tasks run successfully

Task 9 — Create EventBridge Trigger

Create Rule

1. Open EventBridge → Create rule

Name: trigger-ecs-task

Configure Event Pattern

1. Choose AWS events
2. Paste event pattern JSON
3. Update bucket name

Set Target

1. Target → Step Functions state machine
2. Select workflow
3. Create rule

Test Trigger

Upload Non-Trigger Files

1. Upload orders and products
2. Confirm workflow not triggered

Upload Trigger File

1. Upload order_items file

Validate Trigger

1. Open Step Functions
2. Confirm execution started

Outcome

- Docker images stored in ECR
- Validation and ETL applications deployed as ECS tasks
- DynamoDB tables populated with processed metrics
- Step Functions successfully orchestrates workflow
- Files archived after processing
- EventBridge automatically triggers pipeline on file upload
- Fully functional event-driven data pipeline implemented